Git & GitHub

VOM ISSUE ZUM MERGE

PR Software Engineering · Team 01

Felix Kirschner · Bernhard Traintinger

Kathrin Reissner · Julian Günther

16.10.2026

Lokal vs. Remote

Lokal (eigener PC)	Remote (GitHub)
Dateien bearbeiten	Gemeinsamer Code
Commits erstellen	Commits austauschen
Lokale Branches	Pull Requests & Reviews

Merksatz:
`commit` → lokal speichern · `push` → hochladen · `pull` → herunterladen

Lokales und Remote-Repository

Lokal (eigener PC) ↔ Remote (GitHub)

- `git clone`: Remote-Repository auf den PC kopieren
- `git commit`: Änderungen **lokal** speichern
- `git push`: lokale Commits zu GitHub hochladen
- `git pull`: Änderungen von GitHub holen

`origin` bezeichnet üblicherweise das verknüpfte Remote-Repository.

Werkzeuge für Git & GitHub

- **Git CLI:** Befehle im Terminal (PowerShell, CMD, Bash)
- **IDE:** Git-Funktionen direkt in Visual Studio / IntelliJ
- **GitHub Desktop:** grafische Oberfläche für Clone, Branches, Commits, Push und Pull Requests
- **GitHub Web:** Issues, Pull Requests und Reviews

Git CLI herunterladen · **GitHub Desktop herunterladen**

Workflow: vom Issue zum Merge

Issue → Branch → Commit → Push → Pull Request → Review → Merge

- Für Features und Bugs ein **Issue / eine User Story / Task** anlegen
- Änderungen auf `feature/*`, **nicht direkt auf `main`**
- Commits und PRs mit dem Issue verknüpfen
- Merge nach `main` erst nach **Review durch ein Teammitglied**

Git CLI · Clone, Checkout & Branch

```
# Repository lokal kopieren
git clone https://github.com/OWNER/prse-team01-github-
demo.git
cd prse-team01-github-demo

# Neuen Feature-Branch erstellen und wechseln
git switch -c feature/hello-world

# Zwischen bestehenden Branches wechseln
git switch main
# Klassische Alternative: git checkout main
```

`clone` = Repository herunterladen · `checkout` / `switch` = Branch bzw. Arbeitsstand wechseln

Git CLI · Commit, Push & Issue-Referenz

```
git switch feature/hello-world
# demo/Program.cs bearbeiten

git add demo/Program.cs
git commit -m "Feature: add greeting | refs #1 | time:
30m"
git push -u origin feature/hello-world
```

- `refs #1`: Commit verweist auf **Issue / User Story / Task #1**
- `Fixes #1` im **Pull-Request-Text** schließt das Issue beim Merge nach `main`
- `time: 30m`: Beispiel für das in der LV geforderte Zeit-Tag

Pull Request & Review

1. Auf GitHub **Compare & pull request** auswählen
2. Änderungen beschreiben und Issue verlinken: `Fixes #1`
3. Teammitglied prüft den Diff und gibt ein **Review** ab
4. PR nach Freigabe nach `main` **mergen**

```
# Lokalen main nach dem Merge aktualisieren
git switch main
git pull origin main
```

PR = nachvollziehbare Änderung mit Diskussion, Review und Merge-Historie

Merge-Konflikt lösen

Szenario: Zwei Branches ändern dieselbe Zeile in `demo/Program.cs`.

```
git switch feature/hello-world
git fetch origin
git merge origin/main
# Konfliktstellen in Program.cs manuell / in der IDE auflösen
git add demo/Program.cs
git commit
git push
```

Konfliktmarker <<<<<<<, =======, >>>>>>> bereinigen; danach Build prüfen.

GitHub + KI · natürliche Sprache

> *„Lies Issue #1, erstelle einen Feature-Branch und passe `demo/Program.cs` an. Zeige mir den Diff, schlage eine Commit-Message mit Issue-Referenz vor und bereite einen Pull Request vor. Nicht ohne meine Freigabe mergen."*

Möglicher Ablauf mit GitHub Copilot / Agent: Anforderung → Codeänderung → Git-Befehle → PR-Entwurf

Grenzen: Tool-Zugriff, Berechtigungen und Bestätigungen. Mensch prüft Diff, Build und Review.

Live-Demo · C#/.NET

Issue → Clone → Branch → Code → Commit → Push → PR → Review → Merge

Speaker notes